JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번: 202219348 이름: 권준성 **제출일:** 2026/10/28

서비스 이름 : 배부르면 행복합니다! - 맛집 공유 게시판

기술 조합과 버전 : Vue 3.5.43 / Node.js v24.21.0, Express 5.3.0 / SQLite(sql.js 1.14.2)

GitHub 저장소 URL : https://github.com/kwonjunsun9/26WSD-assignment-01.git

소스 코드 기준 : master/49cdb74 (최종 제출 커밋 SHA 는 LMS 에 제출할 예정입니다.)

– 서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

이번 과제에서는 맛집에 대한 정보를 공유하고 싶은 사용자들을 대상으로 맛집 공유 게시판을 구현하였습니다. 사용자는 식당 이름과 위치, 방문 후기를 게시글의 제목과 본문에 작성하여 다른 사용자에게 공유할 수 있습니다. 게시글 목록과 상세 내용은 로그인하지 않아도 볼 수 있도록 하였으며, 글 작성은 로그인한 사용자만 가능하고 수정과 삭제는 작성자 본인만 가능하도록 구성하였습니다.

기술 조합은 Vue 3 와 Node.js Express 를 선택하였습니다. 화면과 API 서버를 모두 JavaScript 로 구성하여 사용자 입력이 서버로 전달되고 결과가 화면에 반영되는 흐름을 연결하였으며, 별도의 DB 서버 없이 게시글을 파일로 저장할 수 있도록 SQLite 를 사용하였습니다.

제공된 템플릿의 기본 게시글 CRUD 구조와 계정의 비밀번호 검증은 유지하고, 미완성 상태였던 로그인과 JWT 발급, 토큰 전송 및 서버 검증을 구현하였습니다. 그리고 로그인한 사용자와 게시글 작성자를 비교하여 수정과 삭제 권한을 검사하고, 토큰 만료나 요청 실패 시에는 원인에 맞는 안내가 화면에 표시되도록 하였습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	frontend-vue/src/App.vue, backend-express/src/app.js	로그인 없이 게시글 목록과 상세 조회를 가능하게 합니다.
로그인과 JWT 발급	완료	frontend-vue/src/api.js, backend-express/src/app.js, backend-express/src/auth.js, backend-express/src/users.js	테스트 계정인 Alice/Bob 을 사용합니다. HS256 JWT 를 발급합니다.
인증과 내 정보 조회	완료	frontend-vue/src/api.js, backend-express/src/auth.js, backend-express/src/app.js	Bearer 토큰 검증 후 ID, 이름, 이메일을 반환합니다.
본인 글 작성·수정·삭제	완료	frontend-vue/src/App.vue, backend-express/src/app.js, backend-express/src/auth.js	서버에서 작성자를 결정하고, 타인의 수정과 삭제는 거부하도록 하였습니다.

기능	완료 여부	관련 파일	비고 또는 제한
만료·로그아웃·오류 표시	완료	frontend-vue/src/api.js, frontend-vue/src/App.vue	인증 실패 시 로그인 상태를 해제하고 원인에 맞는 안내를 표시하였습니다. 토큰 만료는 인증이 필요한 요청을 보낼 때 확인하도록 하였습니다.
DB 데이터 유지	완료	backend-express/src/store.js	SQLite 파일을 저장하고, 서버가 재시작 된 후 게시 글들이 유지되도록 합니다.

구조와 변경 내역

- 브라우저 → API 서버 → SQLite 의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.

전체 구조는 브라우저의 Vue 화면에서 API 를 호출하면, Vite 프록시를 통해 Express 서버로 요청이 전달되는 방식으로 구성하였습니다. 서버에서는 요청에 필요한 인증과 권한을 검사하고, 게시글을 조회하거나 변경할 때는 SQLite 를 사용하도록 하였습니다. 처리 결과는 JSON 으로 반환하여 브라우저의 게시글 목록과 상세 화면에 반영하도록 하였습니다.

로그인할 때는 입력한 이메일과 비밀번호를 POST /api/auth/login 으로 전송합니다. 정상 계정이면 issueToken() 함수를 통해 JWT 를 발급하도록 하였습니다. 브라우저에서는 응답으로 받은 토큰과 사용자 정보를 메모리에 저장하며, localStorage 나 sessionStorage 는 사용하지 않았습니다. 따라서 새로고침하면 로그인 상태는 해제되지만, DB 에 저장한 게시글은 유지되도록 하였습니다.

게시글을 수정할 때는 PUT /api/posts/{id} 요청에 Authorization: Bearer <JWT> 헤더를 포함해서 전송하도록 하였습니다. 서버에서는 currentUserId() 함수로 토큰을 검증한 뒤 게시글을 조회하고, assertOwner() 함수로 로그인한 사용자와 작성자가 같은 지 확인하도록 하였습니다. 이후 제목과 본문을 검사하여 DB 를 수정하고, 반환된 결과를 화면에 반영하였습니다. 인증이나 작성자 권한 검사에 실패하면 DB 를 변경하지 않고 오류를 반환하도록 하였습니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-express/src/auth.js — issueToken(), currentUserId(), assertOwner()	JWT 발급은 미구현 상태였고, 사용자 ID 는 1 을 반환하고 작성자 검사는 수행되지 않았습니다.	환경변수 키를 이용한 HS256 JWT 를 발급받도록 하였습니다. 그리고 토큰을 검증하고, 실제 사용자 확인 및 작성자 비교 구현하였습니다.	데모 사용자 대신 인증된 사용자를 구분하도록 하고, 타인이 글을 변경하는 것을 차단하기 위해서
backend-express/src/app.js — fields(), 오류 처리 미들웨어	로그인/CRUD 라우트, 입력 검사와 오류의 응답 구조를 제공하였습니다.	허용하는 필드와 문자열의 길이 제한의 검사를 보완하였습니다. JSON 처리 관련 오류를 400 응답으로 처리하도록 하였습니다.	잘못된 입력을 거부하고 API 명세에 맞는 오류를 적절하게 반환하도록 하였습니다.
frontend-vue/src/api.js — login(), authorizationHeaders(), request(), canEdit()	공통 API 요청 구조는 제공되었으나 로그인 연결과 토큰 전송이 미완성이었습니다. 수정 가능 여부는 항상 true 를 반환하고 있었습니다.	로그인 응답을 메모리에 저장하도록 하고 Bearer 헤더를 전송하게 하였습니다. 그리고 401 응답 시 세션을 해제하고 작성자가 누구인지 비교하도록 하였습니다.	서버의 인증 상태와 화면의 로그인 상태를 적절하게 연결하기 위해 변경하였습니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
frontend-vue/src/App.vue — run(), save(), logout(), 화면 문구	기본 CRUD 화면과 로그인 입력 창, 공통 오류 표시 구조를 제공하였습니다.	인증 시 오류 별 안내를 제공합니다. 존재하지 않는 글을 조회할 시 화면을 정리하고 목록을 다시 갱신합니다. 그리고 비로그인으로 글쓰기를 제한하고 로그아웃 시 입력이 초기화되도록 하였습니다.	요청한 결과에 맞게 화면을 갱신하고, 사용자가 실패 원인을 파악할 수 있도록 하기 위해서 변경하였습니다.

- 각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

기본 CRUD 라우트와 제공 계정의 비밀번호 검증, SQLite 저장 기능은 제공 코드를 유지하였습니다. 인증과 프론트 연결 부분은 AI 의 제안을 참고하여 코드를 직접 반영하였으며, 실제 API 요청과 화면 동작을 확인하는 방식으로 검증하였습니다

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

실행 환경은 Windows 11 과 Node.js v24.21.0 이며, Naver Whale 브라우저에서 화면과 API 응답을 확인하였습니다.

백엔드는 'backend-express' 폴더에서 'npm start'로 실행하였으며, 주소는 '127.0.0.1:8000' 입니다.

프론트엔드의 빌드는 'frontend-vue' 폴더에서 'npm run build', 빌드 결과의 미리보기 명령은 'npm run preview'로 구성되어 있습니다.

게시글은 기본적으로 백엔드의 data/board.sqlite 파일에 저장하도록 하였습니다.

T16 에서는 127.0.0.1:4173 의 화면에서 게시글 목록 요청이 200 OK 로 응답하는 것을 확인하였습니다. 또한 T15 에서는 게시글을 등록한 뒤 백엔드를 종료하고 다시 실행하여, 같은 게시글을 조회했을 때 작성자와 내용이 유지되는 것을 확인하였습니다. 관련 증거는 docs/evidence/의 T15 와 T16 으로 시작하는 캡처 화면에 정리하였습니다.

핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20 줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

3 1 로그인과 JWT 발급

파일 / 함수: backend-express/src/app.js —POST /api/auth/login (line 50), fields() (line 10)

backend-express/src/auth.js —issueToken() (line 26)

backend-express/src/users.js —제공된 verifyCredentials() (line 16) 사용

코드 발췌

- 로그인 요청 처리 (backend-express/src/app.js) (line 50)

```
app.post("/api/auth/login", (req, res) => {
  const input = fields(req.body, { email: 254, password: 128 });
  const user = verifyCredentials(input.email, input.password);
  if (!user)
    throw new ApiError(
      401,
      "INVALID_CREDENTIALS",
      "이메일 또는 비밀번호를 확인하세요.",
    );
  const accessToken = issueToken(user);
  res.json({
    accessToken,
    expiresIn: Number(process.env.TOKEN_TTL || 90),
    user,
  });
});
```

- JWT 발급 (backend-express/src/auth.js) (line 26)

```
export function issueToken(user) {
  return jwt.sign({}, jwtKey, {
    algorithm: "HS256",
    subject: String(user.id),
    expiresIn: tokenTtl,
    issuer,
    audience,
  });
}
```

동작과 응답 코드 설명

[입력 검증, 계정 확인, 토큰 클레임과 유효 시간, 응답을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.

처음에는 POST /api/auth/login 으로 전달된 이메일과 비밀번호를 fields() 함수를 통해 검사하도록 하였습니다. 이메일은 최대 254 자, 비밀번호는 최대 128 자의 문자열만 허용하며, 입력이 누락되거나 공백만 있는 경우와 추가 필드가 포함된 경우에는 400/BAD_INPUT 을 반환하도록 하였습니다.

입력 검사가 끝나면 제공된 verifyCredentials() 함수로 계정을 확인하였습니다. 이 함수는 입력한 이메일에 일치하는 계정을 찾고, 입력 비밀번호를 scryptSync()를 통해 계산한 뒤 저장된 해시를 timingSafeEqual()로 비교합니다. 따라서 제공된 비밀번호 검증은 유지하면서 정상 계정에 대해서만 토큰이 발급되도록 하였습니다. 존재하지 않는 계정이거나 비밀번호가 일치하지 않으면 401/INVALID_CREDENTIALS 를 반환하고, issueToken()은 호출하지 않도록 하였습니다.

계정이 올바른 지 확인하면 issueToken() 함수에서 jwt.sign()을 사용하여 JWT 를 발급하도록 하였습니다. 서명에 사용하는 jwtKey 는 서버 환경변수의 JWT_SECRET 이 64 자리 16 진수인지 검사한 뒤 이를 32 바이트로 변환한 값입니다. 서명 알고리즘은 HS256 으로 고정하고, subject 에는 사용자 ID 를 문자열로 전달하여 sub 클레임에 기록하도록 하였습니다. 또한 expiresIn 으로 만료 시각인 exp 가 생성되도록 하였으며, 기본 유효 시간은 90 초이고 환경변수로 조절할 수 있도록 하였습니다.

마지막으로 로그인에 성공하면 HTTP 200 OK 메시지와 함께 accessToken, expiresIn, user 를 반환하도록 하였습니다. user 에는 ID, 이름, 이메일만 포함하며, 비밀번호와 비밀번호 해시, 서버 비밀 키는 응답이나 토큰에 넣지 않았습니다. 성공 응답에는 JSON code 필드가 없으며, 실패할 때는 공통 오류 처리 미들웨어에서 코드와 메시지를 반환하게 하였습니다.

상황	HTTP 상태	JSON 코드	응답 및 처리
정상 이메일과 비밀번호로 로그인 (성공)	200	없음	accessToken, expiresIn, user 반환
존재하지 않는 계정 또는 비밀번호 불일치 (실패)	401	INVALID_CREDENTIALS	로그인 실패 안내, 토큰 미발급
입력 누락, 타입, 길이 오류, 공백만 입력 또는 추가 필드 포함 (실패)	400	BAD_INPUT	입력 오류 안내, 계정 확인 전에 요청 거부

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-vue/src/api.js — login() (line 44), authorizationHeaders() (line 8), request() (line 16), clearSession() (line 3)

frontend-vue/src/App.vue — logout() (line 106), run() (line 21)

코드 발췌

- 로그인 응답 저장과 인증 헤더 생성 (frontend-vue/src/api.js)

```
function authorizationHeaders() {
  const token = session.accessToken;

  if (!token) {return {};}

  return {Authorization: `Bearer ${token}`,};
}
```

```
export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });

  session = {
    accessToken: data.accessToken,
    user: data.user,
  };

  return session.user;
}
```

- 로그아웃 처리 (frontend-vue/src/App.vue)

```
function logout() {
  clearSession();
  user.value = null;
  reset();
  message.value = "로그아웃 되었습니다.";
}
```

동작과 응답 코드 설명

로그인에 성공하면 login() 함수에서 서버가 반환한 accessToken 과 user 를 session 변수에 저장하도록 하였습니다. session 은 브라우저의 JavaScript 메모리에 있는 변수이며, localStorage 나 sessionStorage 에는 저장하지 않았습니다. 따라서 페이지를 새로고침하면 토큰과 사용자 정보가 초기화되어 로그인 상태가 해제되도록 하였습니다. 화면에서는 run() 함수의 마지막에 getSession().user 를 확인하여 현재 로그인한 사용자를 표시하도록 하였습니다.

이후 API 를 요청할 때는 authorizationHeaders() 함수에서 저장된 토큰이 있는지 확인하도록 하였습니다. 토큰이 있으면 Authorization: Bearer <JWT> 형태의 헤더를 반환하고, 없으면 빈 객체를 반환하여 인증 헤더를 생략하도록 하였습니다. 공통 request() 함수에서는 이 결과를 fetch()의 요청 헤더에 포함하였습니다. 따라서 토큰이 저장되어 있으면 공통 함수를 사용하는 API 요청에 토큰이 함께 전송되며, 내 정보 조회와 게시글 작성, 수정, 삭제 요청에서도 같은 방식을 사용하도록 하였습니다.

서버에서 HTTP 401 을 반환하면 request() 함수의 response.status == 401 조건에서 clearSession()을 호출하도록 하였습니다. 이 함수는 토큰을 빈 문자열로, 사용자 정보를 null 로 초기화합니다. 이후 서버의 HTTP

상태와 JSON code 를 오류 객체에 담아 전달하고, 화면의 run() 함수에서 로그인 실패, 토큰 누락, 변조 또는 만료에 맞는 안내를 표시하도록 하였습니다. 토큰 만료도 만료된 토큰으로 요청한 뒤 401 TOKEN_EXPIRED 응답을 받았을 때 로그인 상태를 해제하도록 하였습니다. 반면 403 FORBIDDEN 은 로그인 상태를 유지하면서 권한 부족 안내를 표시하도록 하였습니다.

로그아웃 버튼을 누르면 logout() 함수에서 clearSession()을 호출하고 화면의 사용자 정보와 작성 중인 입력을 초기화하도록 하였습니다. 로그아웃은 브라우저 내부 상태를 정리하는 동작이므로 별도의 서버 요청이나 HTTP 응답은 발생하지 않습니다. 로그아웃한 뒤 인증이 필요한 API 를 요청하면 토큰이 없는 상태로 전송되어 401 TOKEN_REQUIRED 를 받게 됩니다. 로그아웃 동작은 이미 복사된 JWT 를 서버에서 즉시 무효화하는 처리는 수행하지 않습니다.

상황	HTTP 상태	JSON 코드	응답 및 처리
로그인 성공	200	없음	토큰과 공개 사용자 정보를 메모리에 저장하고 로그인 상태 표시
정상 토큰으로 내 정보 조회	200	없음	반환된 사용자 정보 표시
계정 또는 비밀번호 불일치	401	INVALID_CREDENTIALS	세션을 해제하고 로그인 실패 안내
토큰 없이 보호 API 요청	401	TOKEN_REQUIRED	세션을 해제하고 로그인 필요 안내
변조 등 유효하지 않은 토큰으로 요청	401	INVALID_TOKEN	세션을 해제하고 재로그인 안내
만료된 토큰으로 요청	401	TOKEN_EXPIRED	세션을 해제하고 로그인 시간 만료 안내
자신이 아닌 다른 작성자의 글 수정, 삭제 요청	403	FORBIDDEN	로그인 상태를 유지하고 권한 부족 안내
화면에서 로그아웃 버튼 클릭	해당 없음	해당 없음	메모리의 토큰·사용자 정보와 화면의 로그인 상태·입력 초기화

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: backend-express/src/auth.js —currentUserId() (line 38)

backend-express/src/users.js —기본 제공된 findPublic() (line 12) 사용

backend-express/src/app.js —/api/me 라우트, 공통 오류 처리 미들웨어

코드 발췌

- 서명, 발급자, 대상 만료 검증 (backend-express/src/auth.js)

```
export function currentUserId(req) {
  const authorization = req.get("Authorization");
  const match =
    typeof authorization === "string"
      ? /^Bearer +(\S+)$/i.exec(authorization)
      : null;
```

```
const options = {
  algorithms: ["HS256"],
  issuer,
  audience,
};

let payload;
let expired = false;

try {
  try {
    payload = jwt.verify(token, jwtKey, options);
  } catch (error) {
    if (!(error instanceof jwt.TokenExpiredError)) {
      throw error;
    }

    // 만료 외에 서명·발급자·대상 오류도 있는지 확인합니다.
    payload = jwt.verify(token, jwtKey, {
      ...options,
      ignoreExpiration: true,
    });
    expired = true;
  }
} catch {
  throw new ApiError(
    401,
    "INVALID_TOKEN",
    "유효하지 않은 토큰입니다.",
  );
}
```

- 실제 사용자 확인, 최종 검증 결과 처리 (backend-express/src/auth.js)

```
if (!payload || typeof payload !== "object" || !Number.isFinite(payload.exp) ||typeof payload.sub !== "string" || !/^[1-9]\d*$/.test
(payload.sub)) {
  throw new ApiError(
    401,
    "INVALID_TOKEN",
    "토큰의 필수 클레임을 확인하세요.",
  );
}

const userId = Number(payload.sub);

if (!Number.isSafeInteger(userId) || !findPublic(userId)) {
  throw new ApiError(
    401,
    "INVALID_TOKEN",
    "사용자를 확인할 수 없습니다.",
  );
}

if (expired) {
  throw new ApiError(
    401,
    "TOKEN_EXPIRED",
    "로그인 시간이 만료되었습니다. 다시 로그인하세요.",
  );
}

return userId;
}
```

동작과 응답 코드 설명

처음에는 currentUserId() 함수에서 Authorization 헤더의 Bearer 형식을 확인하고, 토큰이 없거나 형식이 잘못되면 '401 TOKEN_REQUIRED'를 반환하도록 하였습니다. 이후 jwt.verify()로 서버 키를 이용한 서명을 검증하고, 허용 알고리즘이 'HS256' 인지와 발급자 iss, 사용 대상 aud 가 서버 설정 값과 일치하는지 검사하였습니다.

또한 exp 가 유한한 숫자인지, sub 가 양의 정수 사용자 ID 를 나타내는 문자열인지 확인하고, findPublic()으로 실제 계정의 존재 여부를 검사하였습니다. 서명이나 클레임이 잘못되었거나 계정이 없으면 '401 INVALID_TOKEN'을 반환하도록 하였습니다. 만료 오류가 발생한 경우에는 만료 검사만 제외하여 재검증하고, 나머지 검사까지 통과한 경우에만 '401 TOKEN_EXPIRED'를 반환하여 만료 토큰의 인증을 거부하였습니다.

모든 검사를 통과하면 숫자형 사용자 ID 를 반환하여 라우트에서 사용하도록 하였습니다. /api/me 는 이 ID 로 공개 사용자 정보를 조회하여 HTTP 200 으로 반환하며, 성공 응답에는 JSON code 가 없습니다. 인증 실패 시에는 공통 오류 처리에서 HTTP 401 과 JSON code, message 를 반환하고, WWW-Authenticate: Bearer 헤더를 포함하도록 하였습니다.

상황	HTTP 상태	JSON 코드	응답 및 처리
정상 토큰으로 /api/me 조회 (성공)	200	없음	검증된 사용자 ID 로 공개 사용자 정보를 조회하여 반환
토큰 누락 또는 Bearer 형식 오류 (실패)	401	TOKEN_REQUIRED	인증을 거부하고 토큰 필요 안내
변조·서명·알고리즘·클레임 오류 또는 존재하지 않는 사용자 (실패)	401	INVALID_TOKEN	인증을 거부하고 유효하지 않은 토큰 안내

다른 검사를 모두 통과한 만료 토큰 (실패)	401	TOKEN_EXPIRED	인증을 거부하고 로그인 시간 만료 안내

3.4 작성자 권한 검사

파일 / 함수: backend-express/src/auth.js —assertOwner() (line 113)

backend-express/src/app.js —게시글 등록, 수정, 삭제 라우트, fields() (line 10)

코드 발췌

- 작성자 체크 (backend-express/src/auth.js)

```
export function assertOwner(userId, authorId) {
  if (userId !== authorId) {
    throw new ApiError(403, "FORBIDDEN", "작성자만 수정·삭제할 수 있습니다.");
  }
}
```

- 수정, 삭제 요청에서 권한 검사 (backend-express/src/app.js)

```
app.put("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req),
    id = postId(req),
    p = getPost(id);
  assertOwner(userId, p.authorId);
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
});
app.delete("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req),
    id = postId(req),
    p = getPost(id);
  assertOwner(userId, p.authorId);
  store.remove(id);
  res.status(204).end();
});
```

동작과 응답 코드 설명

게시글을 수정하거나 삭제할 때는 currentUserId()로 인증된 사용자 ID 를 확인하고, getPost()로 DB 의 게시글을 조회하도록 하였습니다. 이후 assertOwner()에서 사용자 ID 와 게시글의 authorId 를 비교하여, 서로 다르면 403 FORBIDDEN 을 반환하도록 하였습니다. 권한 검사는 DB 변경 전에 수행하므로 거부된 요청은 기존 게시글을 변경하지 않도록 하였습니다.

401 은 토큰이 없거나 유효하지 않아 인증에 실패한 경우이며, 403 은 인증에는 성공했지만 해당 글의 작성자가 아닌 경우로 구분하였습니다. 화면에서 수정, 삭제 버튼을 숨기는 것과 별도로 서버에서도 작성자를 검사하여, API 를 직접 호출하는 경우에도 타인의 글을 변경할 수 없도록 하였습니다.

작성자 위조를 방지하기 위해 글 등록 시에는 검증된 사용자 ID 를 서버에서 작성자로 지정하였습니다. 등록, 수정 본문은 title 과 content 만 허용하며, 입력 검사 단계에서 authorId, userId 등의 추가 필드를 거부하도록 하였습니다. 수정할 때도 기존 작성자와 생성 시각은 유지하도록 하였습니다.

상황	HTTP 상태	JSON 코드	응답 및 처리
본인 글 정상 수정 (성공)	200	없음	수정된 게시글 반환, 작성자와 생성 시각 유지
본인 글 정상 삭제 (성공)	204	없음	게시글 삭제, 응답 본문 없음
토큰 누락, 변조, 만료 (실패)	401	TOKEN_REQUIRED / INVALID_TOKEN / TOKEN_EXPIRED	인증 실패로 변경 거부
정상 토큰으로 타인의 글 수정, 삭제 (실패)	403	FORBIDDEN	권한 부족으로 변경 거부, 기존 글 유지
정상 토큰으로 존재하지 않는 글 수정, 삭제 (실패)	404	NOT_FOUND	게시글이 없어 변경 거부
정상 토큰으로 글 등록 시 authorId, userId 추가 (실패)	400	BAD_INPUT	추가 필드 거부, 위조한 글 생성 못함.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: Windows 11 / Node.js v24.21.0 / Naver Whale 5.39.

백엔드는 'npm start'로 실행하였으며, 브라우저 개발자 도구의 Network 와 Console 을 사용하여 API 요청과 응답을 확인하였습니다.

로컬 배포 확인: 새 폴더에 GitHub 저장소를 복제한 뒤, 백엔드와 프론트엔드 폴더에서 각각 'npm ci'를 실행하여 의존성을 설치하였습니다.

백엔드는 환경변수를 설정한 후 'npm start'로 실행하여 '127.0.0.1:8000'에서 동작하도록 하였습니다. 프론트엔드는 'npm run build'를 실행하여 빌드가 정상적으로 완료되고 dist 폴더가 생성되는 것을 확인하였습니다.

이후 백엔드를 실행한 상태에서 'npm run preview'를 실행하고, '127.0.0.1:4173'에 접속하여 빌드된 화면을 확인하였습니다. 개발자 도구의 Network 에서 GET /api/posts 가 200 OK 로 응답하는 것을 확인하여 프론트엔드와 백엔드의 API 연결을 검증하였습니다.

테스트 일시 및 코드 버전: 2026/10/27~2026/10/28(KST) / main 브랜치 / 테스트 결과 정리 기준 커밋 3dcf93a

토큰 TTL: 일반 테스트는 90 초, 만료 테스트인 T10 은 5 초로 설정하였습니다. 만료 테스트에서는 토큰 발급 후 7 초를 대기한 뒤 보호 API 를 요청하여 만료 응답을 확인하였습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	로그아웃 상태에서 토큰이 없는 상태. 1. 목록 조회 후 2. 기존 글의 ID 값 조회	두 요청 모두 200, 목록과 게시글 상세 정상 표시	1. 목록 조회 GET /api/posts → 200 OK 2. 게시글 상세 조회 GET /api/posts/1 → 200 OK	PASS	docs/evidence/T01*.png — 해당 폴더에서 T01 로 시작하는 모든 PNG 파일
T02	Alice 계정의 정상 이메일과 비밀번호로 로그인	200, JWT 와 공개 사용자 정보 반환, 화면에 로그인 상태 표시	POST /api/auth/login → 200 OK, JSON code 없음. accessToken 발급 expiresIn 90 초 Alice(id 1) 정보 반환 화면에 Alice 로그인 상태와 로그아웃 버튼 표시	PASS	docs/evidence/T02*.png — 해당 폴더에서 T02 로 시작하는 모든 PNG 파일

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T03	로그아웃 상태에서 1. Alice 계정에 잘못된 비밀번호로 로그인 2. 존재하지 않는 아이디 kwonjunsung@class.test 로 로그인	각각 401 INVALID_CREDENTIALS, 토큰 미발급, 로그인 실패 안내	POST /api/auth/login 1. 잘못된 비밀번호로 로그인: 401/INVALID_CREDENTIALS 2.존재하지 않는 아이디로 로그인: 401/INVALID_CREDENTIALS 두 응답 모두 토큰 미발급. 실패 안내 표시 및 로그아웃 상태 유지	PASS	docs/evidence/T03*.png — 해당 폴더에서 T03 으로 시작하는 모든 PNG 파일
T04	로그아웃 후 토큰이 없는 상태에서 1. 내 정보 조회 2. Console 의 fetch 를 통하여 글 등록 요청	각각 401 TOKEN_REQUIRED	1. GET /api/me → 401 / TOKEN_REQUIRED 2. POST /api/posts → 401 / TOKEN_REQUIRED 두 요청 모두 Authorization 없음. 응답에 WWW-Authenticate: Bearer 포함. 화면의 로그인 필요 안내 및 로그아웃 상태 확인	PASS	docs/evidence/T04*.png — 해당 폴더에서 T04 으로 시작하는 모든 PNG 파일
T05	Alice 로 1.로그인 후 토큰으로 글 작성 2. 내 정보 조회	글 등록은 201 authorId = 1 /me 는 Alice	1. POST /api/posts → 201 id = 4, authorId = 1, authorName = Alice, Location: /api/posts/4. 등록한 글의 목록, 상세 표시 확인 2. GET /api/me → 200 id = 1, name = Alice, email = alice@class.test 반환.	PASS	docs/evidence/T05*.png — 해당 폴더에서 T05 으로 시작하는 모든 PNG 파일

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T06	Alice 의 토큰으로 본인 글 ID 4 의 본문 수정 후 삭제. 삭제 후 목록 재조회	수정 200 삭제 204 수정 내용 반영 및 삭제한 글 제거	1. 본문 수정 PUT /api/posts/4 → 200 authoId = 1 createdAt 유지 updatedAt 갱신. 2. 본문 삭제 DELETE /api/posts/4 → 204 삭제 후 목록 재조회에서 ID 1 과 2 만 반환되어 ID 4 제거 확인. 화면에 삭제 완료 안내 표시	PASS	docs/evidence/T06*.png — 해당 폴더에서 T06 으로 시작하는 모든 PNG 파일
T07	Bob 의 정상 토큰으로 Alice 의 글 ID 1 을 콘솔에서 수정 요청. 거부 후 글 상세 재 조회	403 FORBIDDEN, 원문 유지	PUT /api/posts/1 → **403 / FORBIDDEN**. 글 상세 재조회 결과: 제목, 본문, authorId, authorName, createdAt, updatedAt 모두 수정 전과 동일하였음. Bob 로그인 표시 유지, 타인의 글 수정과 삭제 버튼 숨겨짐을 확인함.	PASS	docs/evidence/T07*.png — 해당 폴더에서 T07 으로 시작하는 모든 PNG 파일
T08	Bob 의 정상 토큰으로 Alic e 의 글 ID 1 을 콘솔에서 삭제 요청. 거부 후 글 상세 재 조회	403 FORBIDDEN, 글 유지	DELETE /api/posts/1 → 403 / FORBIDDEN. 글 상세 재조회 결과: 글이 정상적으로 존재하며 제목, 본문, authorId, authorName, createdAt, updatedAt 모두 삭제 시도 전과 동일하였음.	PASS	docs/evidence/T08*.png — 해당 폴더에서 T08 으로 시작하는 모든 PNG 파일

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T09	Alice 의 유효 토큰에서 서명 부분 첫 글자만 변조함. /me 및 글 POST·PUT·DELETE 요청. 각 변경 요청 후 목록, 글 상세 재 조회	모두 401 INVALID_TOKEN, 데이터 변경 없음	GET /api/me → 401 / INVALID_TOKEN POST /api/posts → 401 / INVALID_TOKEN PUT /api/posts/1 → 401 / INVALID_TOKEN DELETE /api/posts/1 → 401 / INVALID_TOKEN 모든 응답에 WWW-Authenticate: Bearer 포함. 각 변경 요청 후 목록 일치 확인함. 글 미생성 및 기존 글 내용, 작성자, 시간 정보 유지 확인함.	PASS	docs/evidence/T09*.png — 해당 폴더에서 T09 으로 시작하는 모든 PNG 파일
T10	TTL = 5 초로 Alice 의 토큰을 발급하였음. 정상인 /me 조회 후 7 초를 대기하여 /me, POST, PUT, DELETE 요청함. 화면에서 TTL 만료 후 내 정보를 조회함.	각 API 401 TOKEN_EXPIRED, 데이터 변경 없음. 화면 로그인 해제 및 만료 안내	GET /api/me → 401 / TOKEN_EXPIRED POST /api/posts → 401 / TOKEN_EXPIRED PUT /api/posts/1 → 401 / TOKEN_EXPIRED DELETE /api/posts/1 → 401 / TOKEN_EXPIRED 모든 응답에 WWW-Authenticate: Bearer 포함됨. 변경 후 목록과 원문이 그대로 유지됨. 화면에 로그아웃 상태, 로그인 시간 만료 안내 표시, 로그아웃 버튼 사라짐.	PASS	docs/evidence/T10*.png — 해당 폴더에서 T10 으로 시작하는 모든 PNG 파일
T11	Alice 로 로그인 후 내 정보 정상 조회 확인. 화면에서 로그아웃한 뒤 다시 내 정보 조회	로그아웃 후 Authorization 헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	1. 로그아웃 전 GET /api/me → 200, Authorization: Bearer 포함됨. 2. 로그아웃 후 GET /api/me → 401 / TOKEN_REQUIRED, Authorization 없음. 응답에 WWW-Authenticate: Bearer 포함. 로그인 필요 안내와 로그아웃 상태 확인	PASS	docs/evidence/T11*.png — 해당 폴더에서 T11 으로 시작하는 모든 PNG 파일

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T12	Bob 로그인 상태로 글 등록 요청에 1. authorId:1 추가 2. userId:1 추가	각각 400 BAD_INPUT, 위조 글 미생성	POST /api/posts 1. authorId:1 추가 → 400/BAD_INPUT 2. userId:1 추가 → 400/BAD_INPUT 두 검사 모두 목록 유지함. 새 글 미생성 및 기존 글 데이터 유지 확인	PASS	docs/evidence/T12*.png — 해당 폴더에서 T12 으로 시작하는 모든 PNG 파일
T13	Bob 의 정상 토큰으로 빈 제목, 공백 본문, 제목 101 자, 본문 5,001 자를 각각 등록 및 본인 글 ID 2 수정 요청. 각 요청 후 공개 목록·상세 재조회	모든 요청 400 BAD_INPUT, DB 변경 없음	POST /api/posts 및 PUT /api/posts/2 에서 총 8 개 조건 모두 400/BAD_INPUT. 모든 요청의 목록과 원문 유지 확인함.	PASS	docs/evidence/T13*.png — 해당 폴더에서 T13 으로 시작하는 모든 PNG 파일
T14	존재하지 않는 양의 정수 ID 999999 의 게시글 조회. Bob 의 정상 토큰으로 해당 게시글 수정, 삭제 요청. 잘못된 ID abc 로 게시글 조회.	없는 게시글은 404/NOT_FOUND, 잘못된 ID 는 400/BAD_INPUT. 수정· 삭제에는 정상 토큰 사용	조회, 수정, 삭제 각각 404/NOT_FOUND. abc 조회는 400/BAD_INPUT. 수정, 삭제 전후 목록 유지 확인함.	PASS	docs/evidence/T14*.png — 해당 폴더에서 T14 으로 시작하는 모든 PNG 파일
T15	Bob 로그인 후 홈페이지에서 글 등록. 백엔드 서버를 종료하고 npm start 로 재실행한 뒤 같은 게시글 조회함.	기존 글과 작성자· 내용 유지	등록 201 Created, Location: /api/posts/5. 재시작 후 GET /api/posts/5 는 200 OK. 게시글 데이터 동일. 성공 응답에 code 없음.	PASS	docs/evidence/T15*.png — 해당 폴더에서 T15 으로 시작하는 모든 PNG 파일
T16	Alice 로 로그인한 뒤 브라우저 새로고침함. 로그인 상태와 공개 게시글 목록 확인.	공개 글 유지, 메모리 로그인 상태 해제	로그인 시 200 OK. 새로고침 후 GET /api/posts 는 200 OK, Authorization 없음. 로그아웃 상태 표시, 로그아웃 버튼 제거, 목록에 게시글 3 개 그대로 유지. 성공 응답에 code 없음.	PASS	docs/evidence/T16*.png — 해당 폴더에서 T16 으로 시작하는 모든 PNG 파일

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T17	토큰 없이 ID 5 수정, 삭제 요청함. Bob 으로 로그인해 새 글 ID 6 등록, 수정, 삭제.	무토큰 요청은 401/TOKEN_REQUIRED, 데이터 변경 없음. Bob 의 정상 등록, 수정, 삭제는 각각 201/200/204.	1. 무토큰 PUT, DELETE 401/TOKEN_REQUIRED, 원문 유지 확인함. 2. Bob 으로 로그인 후 등록 201 Location: /api/posts/6, 수정 200, 삭제 204. 삭제 후 목록에서 ID 6 제외됨. 성공 응답에 code 없음.	PASS	docs/evidence/T17*.png — 해당 폴더에서 T17 으로 시작하는 모든 PNG 파일

HTTP 응답 확인 기록

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	토큰 없이 GET /api/posts, GET /api/posts/1 요청 시 각각 200 / code 없음, Alice 의 정상 계정으로 POST /api/auth/login 요청 시 200/code 없음, Alice 의 정상 토큰으로 PUT /api/posts/4 요청 시 200/code 없음	docs/evidence/T01*.png, T02*.png, T06*.png — 해당 폴더에서 T01, T02, T06 로 시작하는 모든 PNG 파일
새 글 등록	**201 및 Location**	POST /api/posts → 201 / code 없음. 응답 헤더에서 Location: /api/posts/4 확인	docs/evidence/T05*.png — 해당 폴더에서 T05 로 시작하는 모든 PNG 파일
본인 글 삭제	**204 및 빈 본문**	DELETE /api/posts/4 → 204/응답 본문 없음. 삭제 완료 안내와 목록 갱신 확인	docs/evidence/T06*.png — 해당 폴더에서 T06 로 시작하는 모든 PNG 파일
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	빈 제목, 공백 본문, 길이 초과의 등록, 수정 요청 및 authorId, userId 를 추가한 등록 요청에서 400/BAD_INPUT	docs/evidence/T12*.png, T13*.png — 해당 폴더에서 T12, T13 으로 시작하는 모든 PNG 파일
잘못된 계정	**401 INVALID_CREDENTIALS**	잘못된 비밀번호와 존재하지 않는 계정으로 로그인 시 401/INVALID_CREDENTIALS, 토큰 미발급	docs/evidence/T03*.png — 해당 폴더에서 T03 로 시작하는 모든 PNG 파일
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	토큰 없이 보호 API 요청 시 401/TOKEN_REQUIRED. 개발 단계의 Authorization: abc 요청도 동일한 응답 확인	docs/evidence/T04*.png, T17*.png — 해당 폴더에서 T04, T17 로 시작하는 모든 PNG 파일
변조·클레임 오류	**401 INVALID_TOKEN**	서명을 변조한 토큰으로 /api/me 및 등록, 수정, 삭제 요청 시 모두 401/INVALID_TOKEN	docs/evidence/T09*.png — 해당 폴더에서 T09 로 시작하는 모든 PNG 파일

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	TTL 5 초로 발급 후 7 초를 대기하여 /api/me 및 등록, 수정, 삭제 요청 시 모두 401/TOKEN_EXPIRED	docs/evidence/T10*.png — 해당 폴더에서 T10 로 시작하는 모든 PNG 파일
다른 작성자의 변경	**403 FORBIDDEN**	Bob 의 정상 토큰으로 Alice 글에 PUT /api/posts/1, DELETE /api/posts/1 요청 시 각각 403 / FORBIDDEN. 재조회 결과 원문 유지	docs/evidence/T07*.png, T08*.png — 해당 폴더에서 T07, T08 로 시작하는 모든 PNG 파일
없는 글	**404 NOT_FOUND**	GET /api/posts/999999 및 Bob 의 정상 토큰을 사용한 해당 글의 수정, 삭제 요청에서 각각 404/NOT_FOUND	docs/evidence/T14*.png — 해당 폴더에서 T14 로 시작하는 모든 PNG 파일

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

토큰 누락, 서명 변조, 만료로 보호 API 가 401 을 반환했을 때 응답 헤더에 WWW-Authenticate: Bearer 가 포함되는 것을 확인하였습니다. 각각 T04, T09, T10 의 헤더 캡처에 기록하였습니다. 등록 성공 시에는 Location: /api/posts/4 를 확인하였으며, 삭제 성공 시에는 204 와 빈 응답 본문을 확인하였습니다.

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

T10 에서 만료된 토큰으로 화면의 내 정보 조회를 요청하자 로그인 상태가 해제되고, 로그아웃 버튼이 사라지며 로그인 시간 만료 안내가 표시되는 것을 확인하였습니다. 반면 T07, T08 에서는 Bob 이 Alice 의 글을 직접 수정, 삭제하여 403 을 받았지만 Bob 의 로그인 상태는 유지되었습니다. 타인의 수정, 삭제 버튼이 숨겨진 상태에서도 직접 API 요청을 보내 서버가 변경을 거부하는 것을 확인하였습니다.

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

Bob 의 수정, 삭제 요청이 거부된 뒤 GET /api/posts/1 로 게시글을 다시 조회하였습니다. 제목, 본문, 작성자와 생성, 수정 시각이 요청 전과 동일하여 기존 글이 변경되지 않았음을 확인하였습니다. 토큰 변조, 만료와 잘못된 입력을 사용한 변경 요청에서도 공개 목록과 상세 내용을 다시 조회하여 데이터가 유지되는 것을 확인하였습니다.

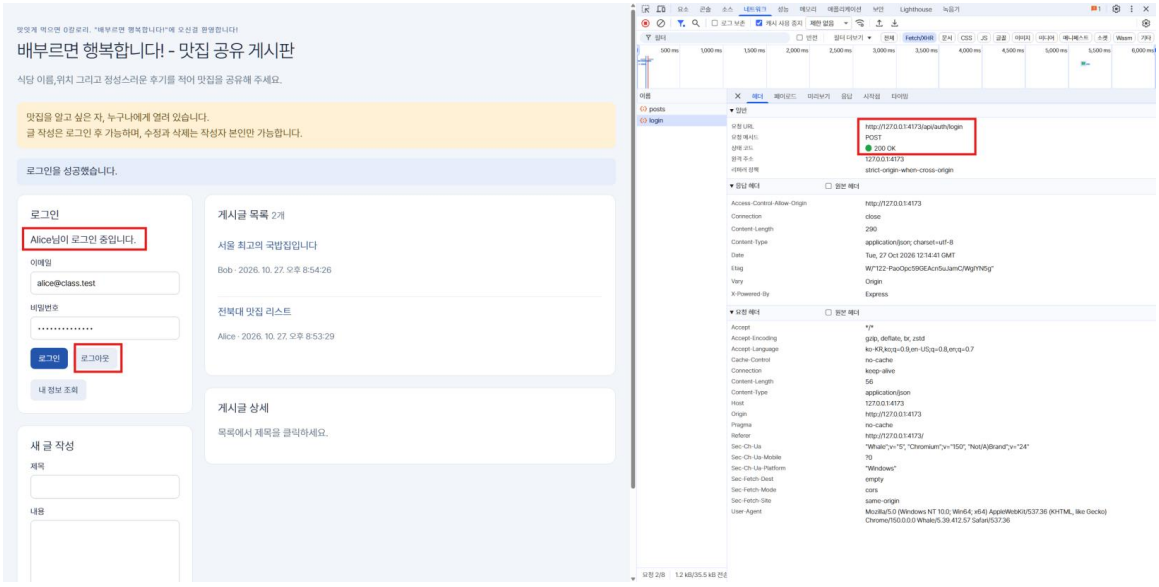
[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

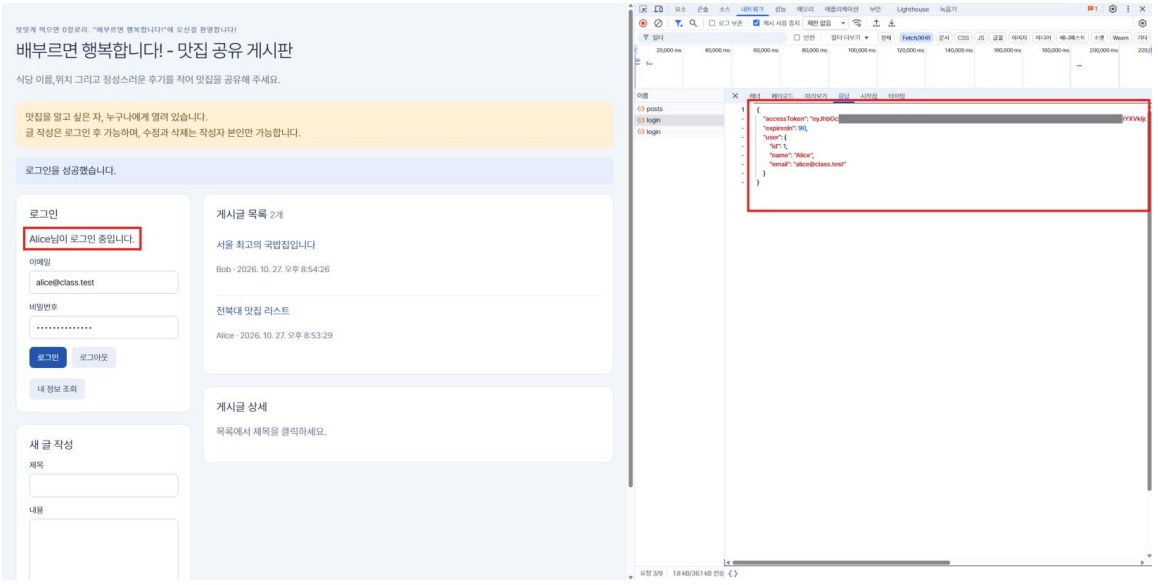
존재하지 않았던 게시글에 토큰 없이 PUT /api/posts/4 요청을 보낸 결과 401/TOKEN_REQUIRED 가 반환되었습니다. 이후 Bob 의 정상 토큰으로 같은 게시글에 수정 요청을 보내자 404/NOT_FOUND 가 반환되었습니다. 이를 통해 게시글의 존재 여부를 확인하기 전에 인증을 먼저 수행하는 것을 확인하였습니다. 두 요청 모두 거부되었으며, 요청 전후 기존 데이터도 동일하게 유지되었습니다.

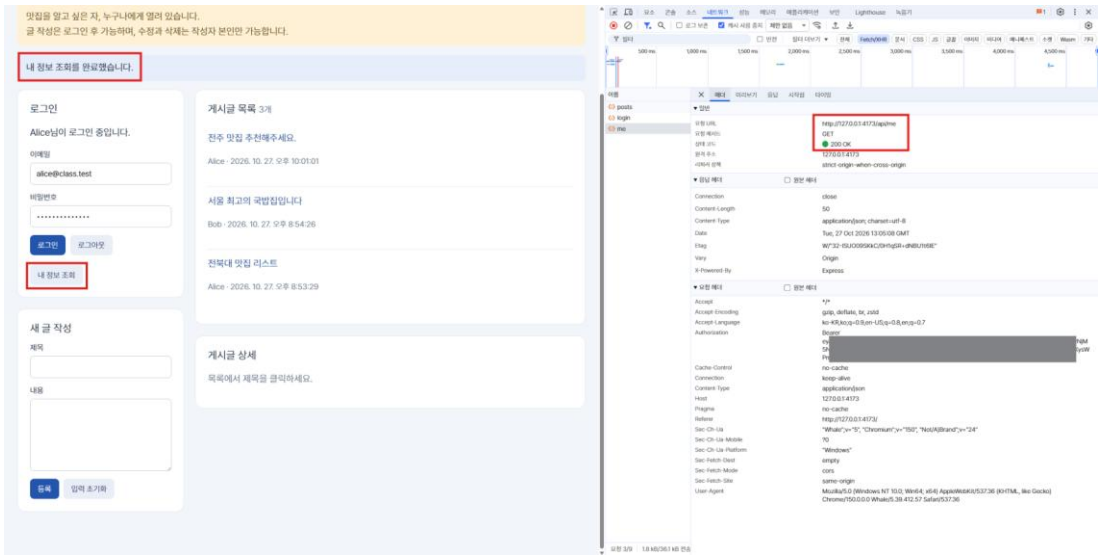
테스트 증거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로그인과 토큰 전송

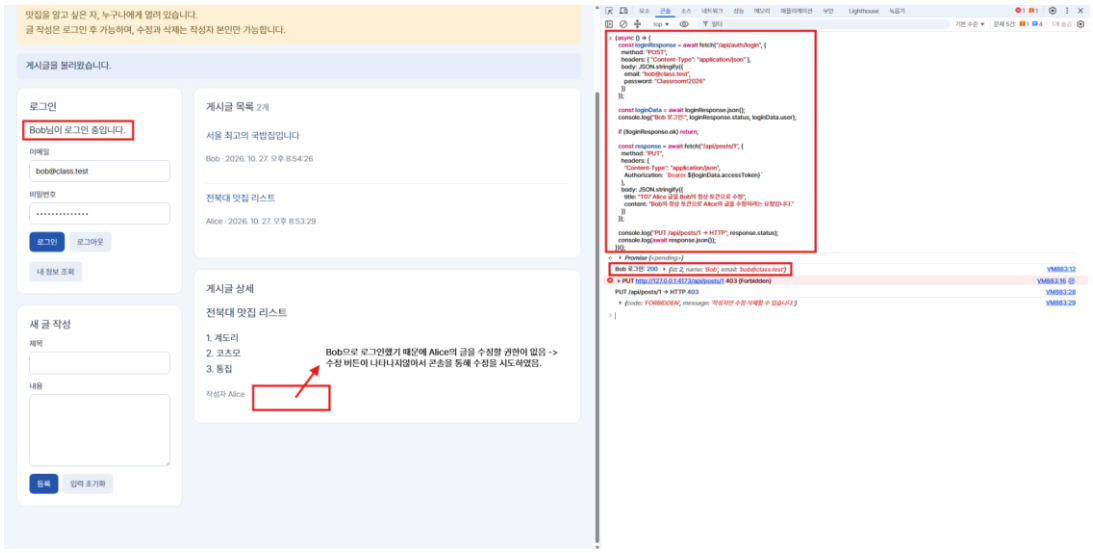


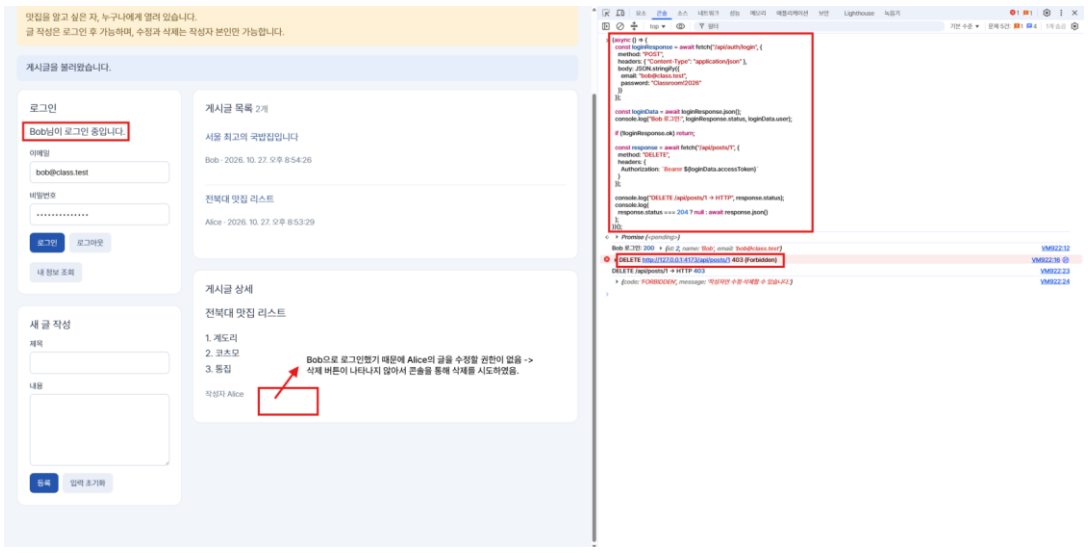




Alice 의 정상 계정으로 로그인하여 HTTP 200 과 함께 accessToken, expiresIn: 90, 공개 사용자 정보가 반환되는 것을 확인하였습니다. 이후 GET /api/me 요청의 Authorization 헤더에 Bearer 토큰이 포함되고, 서버가 200 OK 로 응답하는 것을 확인하였습니다. 캡처에서는 토큰 원문을 가렸습니다.

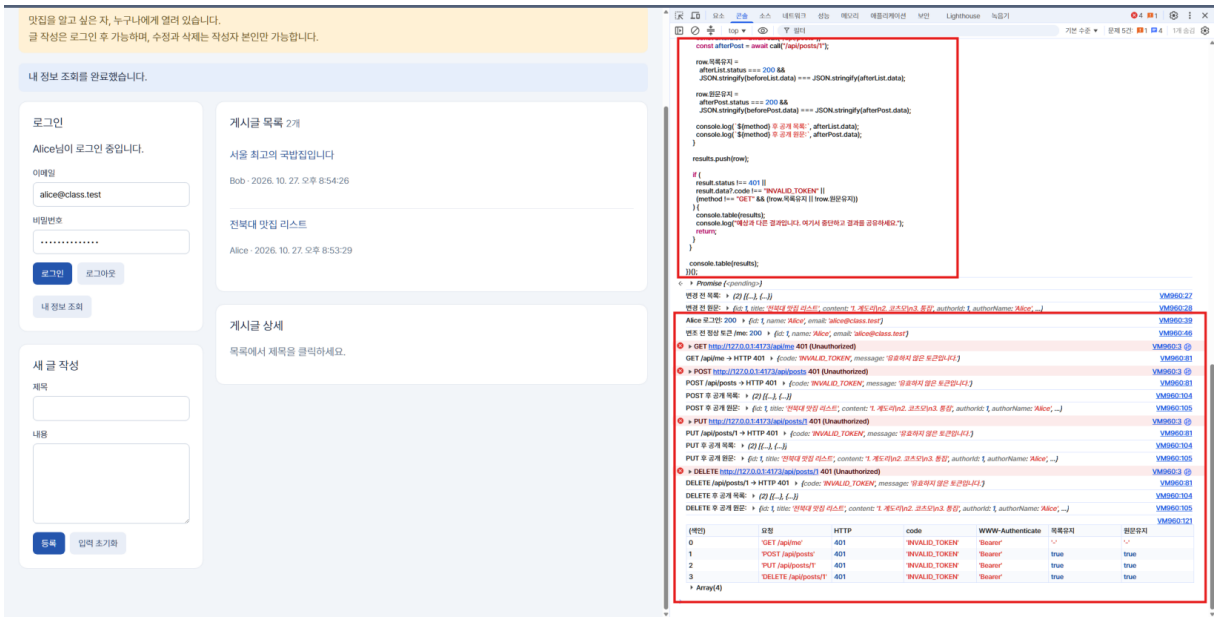
다른 작성자의 수정과 삭제

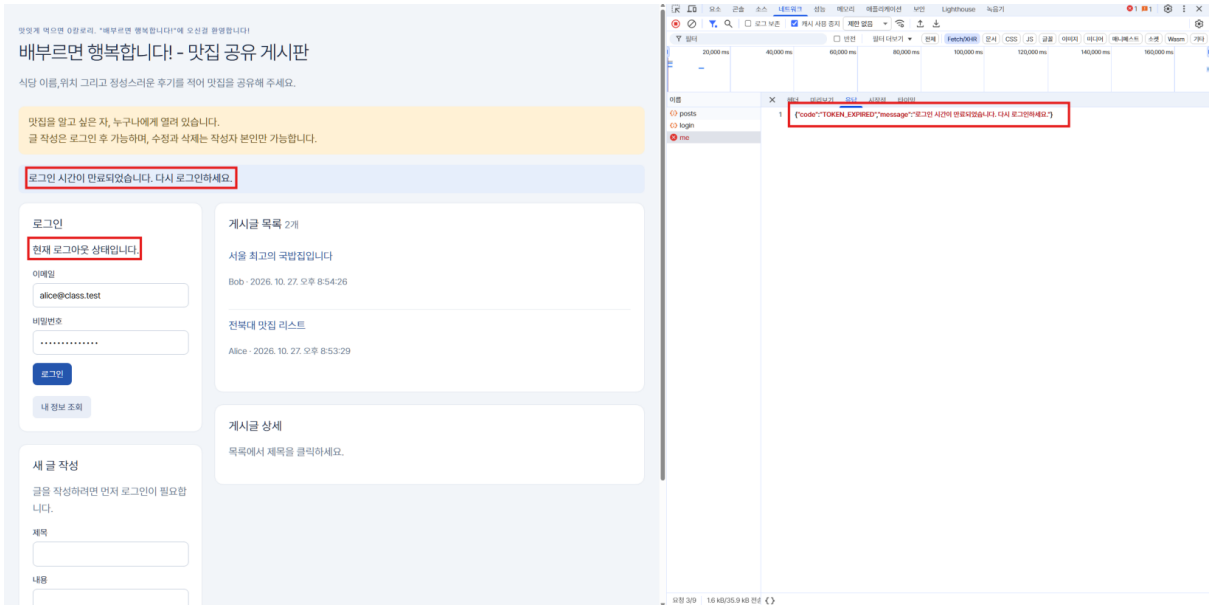




Bob 의 정상 토큰으로 Alice 의 게시글에 수정과 삭제 요청을 직접 전송하였으며, 두 요청 모두 '403 FORBIDDEN'으로 거부되는 것을 확인하였습니다. 화면에서 수정, 삭제 버튼이 숨겨진 상태에서 콘솔을 통해 요청하여 서버의 작성자 검사가 수행되는 것을 확인하였으며, 요청이 거부된 뒤에도 Bob 의 로그인 상태는 유지되었습니다.

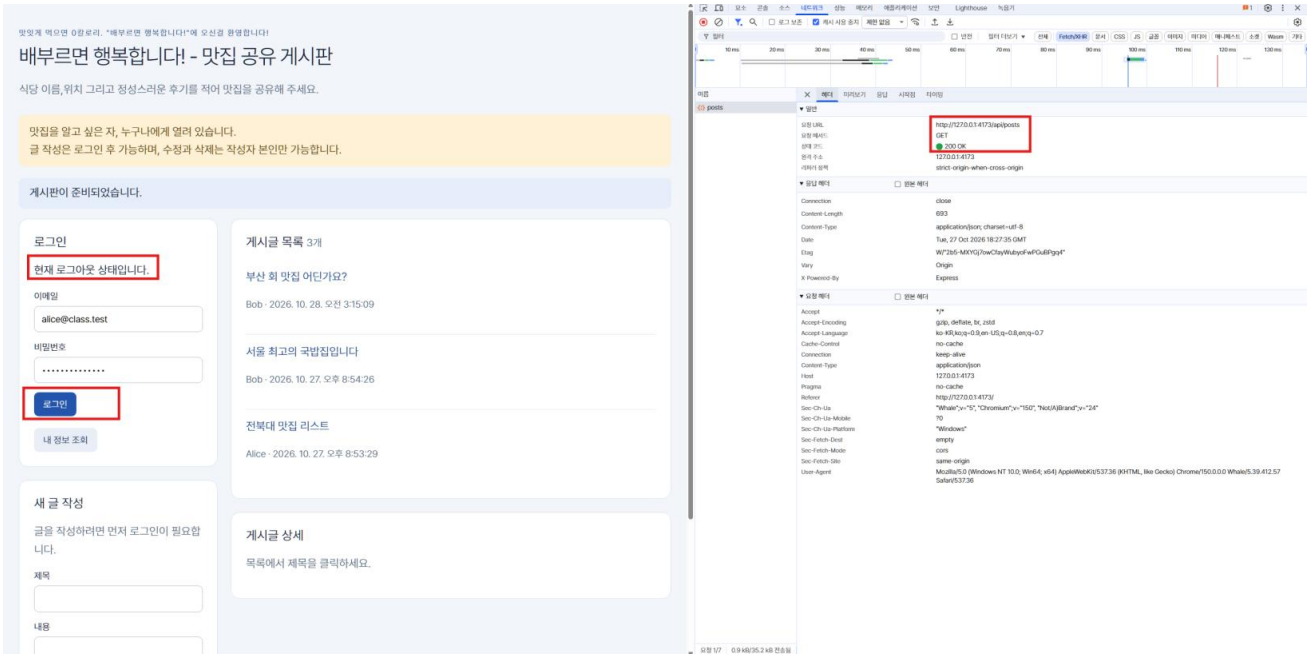
토큰 변조와 만료

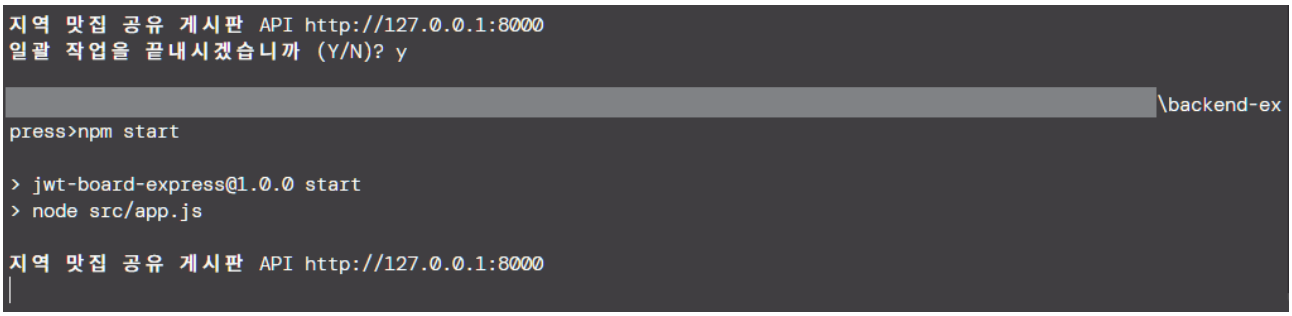


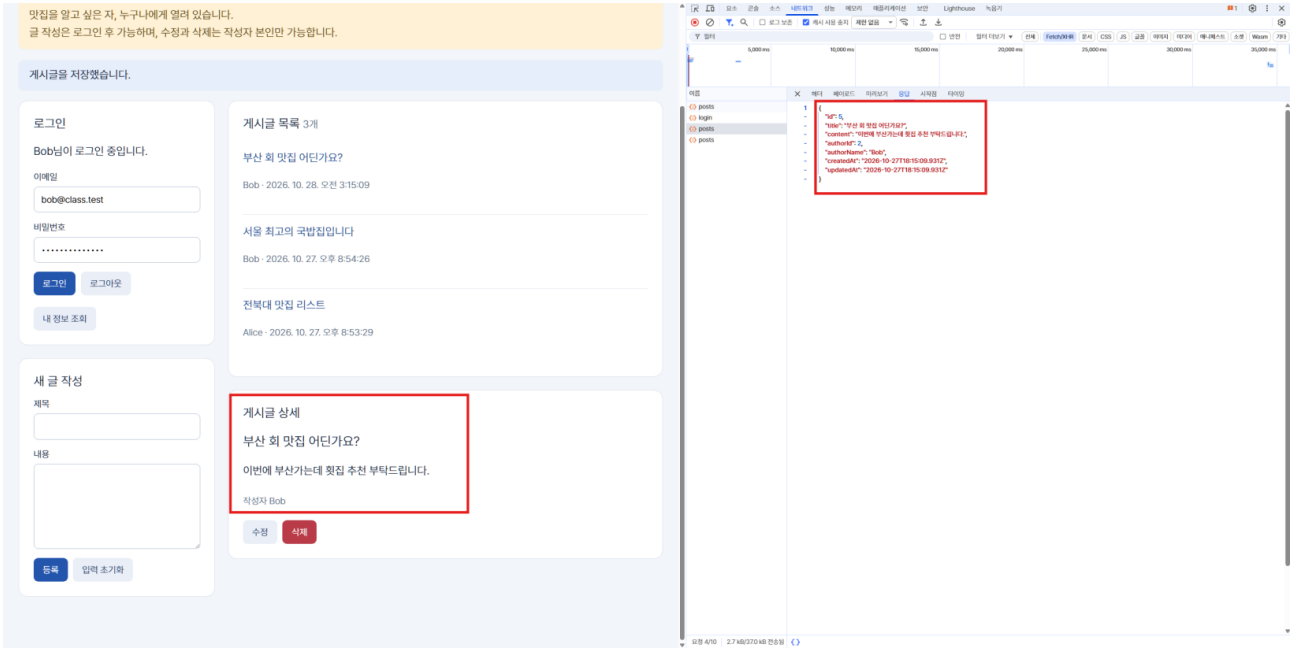


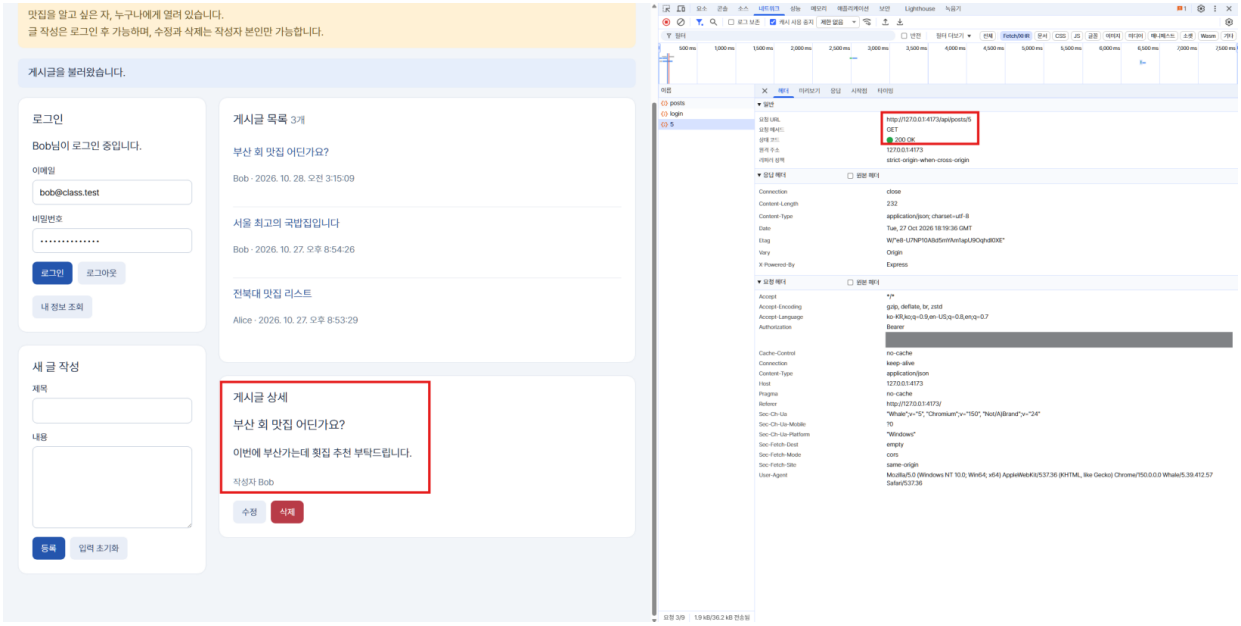
정상 토큰의 서명 일부를 변조한 뒤 내 정보 조회와 게시글 등록, 수정, 삭제를 요청한 결과, 모두 '401 INVALID_TOKEN'으로 거부되고 기존 목록과 원문이 유지되는 것을 확인하였습니다. 또한 만료된 토큰으로 화면의 내 정보 조회를 요청하자 '401 TOKEN_EXPIRED'가 반환되었으며, 로그인 상태가 해제되고 로그인 시간 만료 안내가 표시되는 것을 확인하였습니다.

자동 검사와 로컬 배포 증거









127.0.0.1:4173 에서 로그아웃 상태에서도 게시글 목록 요청이 200 OK 로 응답하는 것을 확인하였습니다. 또한 백엔드를 종료한 뒤 'npm start'로 다시 실행하고, 원본 전후 응답을 확인해 보니 기존 게시글인 GET /api/posts/5 를 조회하여 200 OK 를 확인할 수 있었습니다. 그리고 게시글 ID 5 의 제목·내용·작성자와 생성·수정 시각이 동일해서 작성자와 내용이 유지되는 것을 확인하였습니다.

```
PS                                                  \assignment-01> python .\tools\check_api.py --mode final
PASS health
PASS bad encoded ID
PASS oversized JSON
PASS password raw length
PASS public list
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 78 checks passed (final). UI and restart tests must be performed separately.
```

백엔드가 실행된 상태에서 PowerShell 터미널에서 'python .\tools\check_api.py --mode final'을 실행하였습니다. 공개 조회, 로그인과 인증, 작성자 권한, 입력 검증 및 게시글 변경 요청을 검사한 결과, 최종 출력에서 일반 자동 검사 78 개 항목이 모두 통과한 것을 확인하였습니다.

```
PS C:\Users\KwonJunSung\Desktop\Develop\kjs_git\Univ_Assignment\2026-3-2\Webservice-Design\assignment-01> python .\tools\check_api.py --mode final --expiry
PASS health
PASS bad encoded ID
PASS oversized JSON
PASS password raw length
PASS public list
```

```
PASS login alice@class.test
Waiting 4 seconds for expiration
PASS expired GET /me
PASS expired POST /posts
PASS expired PUT /posts/25
PASS expired DELETE /posts/25
PASS failed request leaves data unchanged
PASS login alice@class.test
PASS login bob@class.test
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 86 checks passed (final). UI and restart tests must be performed separately.
```

이후 토큰의 만료 처리를 확인하기 위해 TOKEN_TTL 을 3 초로 변경하고 백엔드를 다시 실행하였습니다. 같은 위치에서 'python .\tools\check_api.py --mode final --expiry'를 실행하였으며, 토큰 발급 후 4 초를 기다린 뒤 내 정보 조회와 게시글 등록, 수정, 삭제 요청이 각각 401/TOKEN_EXPIRED 로 거부되는 것을 확인하였습니다. 실패한 요청 이후 데이터가 유지되었고, 검사에 사용한 임시 게시글도 정상적으로 삭제되었습니다.

최종 결과는 일반 검사를 포함하여 86 개 항목 모두 통과하였습니다.

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 사용

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
ChatGPT	학습/설계/테스트	auth.js(backend-express/src/auth.js), api.js(frontend-vue/src/api.js), App.vue 콘솔 테스트 코드	설명과 예시 코드를 참고하여 조건에 맞게 적용하였습니다. 잘못된 부분이 있으면 다시 물어봐 코드의 역할을 제대로 알아가며 수정하며 진행하였습니다. 마지막으로 요청의 응답과 화면 변화를 확인하여 검증하였습니다.

사례 1: 데모 사용자 ID 를 실제 JWT 검증 결과로 바꾸는 과정

도구 / 날짜 : ChatGPT / 2026-10-25

요청한 내용: 로그인 API 에서 토큰이 발급되는 것을 확인한 뒤, 해당 토큰이 서버에서 사용자 확인에 어떻게 사용되는지 코드를 살펴보았습니다. 이 과정에서 currentUserId()가 항상 사용자 ID 1 을 반환하는 부분을 수정해야 한다는 점을 확인하였습니다. 다만 토큰에서 사용자 ID를 읽는 것과 토큰의 유효성을 검증하는 것의 차이를 이해하는 데 어려움이 있어, 필요한 검증 항목과 처리 순서를 설명해 달라고 요청하였습니다.

AI 가 제안한 내용: 요청 헤더에서 토큰을 추출하고, jwt.verify()로 서명, 알고리즘, 만료, 발급자, 대상을 순서대로 검증하도록 안내하였습니다. 이후 필수 클레임과 실제 계정의 존재 여부를 확인하고, 검증된 사용자 ID 를 반환하는 코드를 제안하였습니다.

실제 반영 위치: auth.js(backend-express/src/auth.js)의 currentUserId()에 반영하였습니다.

내가 확인하고 수정한 내용: AI 의 설명과 예시를 기존 코드와 비교하며, 서명과 만료 시간뿐만 아니라 발급자, 대상과 필수 클레임도 확인해야 하는 이유를 정리하였습니다. 이를 바탕으로 고정 ID 반환 부분을 수정하고, 검증된 토큰의 사용자 ID를 반환하도록 적용하였습니다. 또한 토큰 누락, 유효하지 않은 토큰과 정상 서명의 만료 토큰이 서로 다른 오류로 처리되는지 코드를 확인하고 실제 요청으로 검증하였습니다.

검증 방법과 결과: T05 에서 정상 토큰의 내 정보 조회가 200 으로 응답하는 것을 확인하였습니다. T04·T09·T10 에서는 토큰 누락·변조·만료가 각각 401 TOKEN_REQUIRED, 401 INVALID_TOKEN, 401 TOKEN_EXPIRED 로 구분되는 것을 확인하였습니다.

사례 2 : 서버의 로그인 응답을 프론트엔드에 연결하는 과정

도구 / 날짜: ChatGPT / 2026-10-26

요청한 내용: 백엔드의 정상 로그인 응답을 확인한 뒤, 프론트엔드의 request()와 session, 화면에서 사용자 정보를 읽는 흐름을 살펴보았습니다. 서버에서 받은 토큰과 사용자 정보를 기존 구조에 어떻게 연결해야 하는지 정리하는 데 어려움이 있어, 각 값의 역할과 로그인 함수에서 수정할 부분을 설명해 달라고 요청하였습니다.

AI 가 제안한 내용: login()에서 로그인 API 를 호출하고, 성공 응답의 accessToken 과 user 를 기존 session 변수에 저장하도록 제안하였습니다. 또한 저장한 사용자 정보가 기존 화면의 세션 조회 흐름을 통해 로그인 표시에 반영되는 과정을 설명하였습니다.

실제 반영 위치: api.js(frontend-vue/src/api.js)의 login()과 App.vue(frontend-vue/src/App.vue)의 로그인 처리 흐름에 반영하였습니다.

내가 확인하고 수정한 내용: 제안된 코드에서 accessToken 은 이후 요청의 인증에 사용되고, user 는 화면의 로그인 표시에 사용된다는 점을 확인하였습니다. 이후 기존 메모리 세션 구조를 활용하여 login()의 미완성 부분을 수정하고, 화면의 사용자 정보 조회 흐름과 연결하였습니다. 적용 후에는 Network 에서 실제 인증 헤더를 확인하였으며, 401 응답에서는 로그인 상태가 해제되고 403 응답에서는 유지되는지도 확인하였습니다.

검증 방법과 결과: T05 에서 Bearer 토큰이 포함된 내 정보 조회가 200 으로 응답하였고, T11 에서는 로그아웃 후 인증 헤더가 생략되어 401 TOKEN_REQUIRED 가 반환되었습니다. T10 에서는 만료 응답 후 화면의 로그인 상태가 해제되었으며, T07·T08 의 403 FORBIDDEN 에서는 Bob 의 로그인 상태가 유지되었습니다.

사례 3 : 콘솔에서 작성자 권한 테스트 요청을 작성하는 과정

도구 / 날짜: ChatGPT / 2026-10-27

요청한 내용: Bob 으로 Alice 의 글을 조회하여 수정· 삭제 버튼이 표시되지 않는 것을 확인하였습니다. 이후 서버에서도 변경 요청을 거부하는지 확인하기 위해 콘솔에서 직접 요청을 보내려고 하였습니다. 다만 fetch()의 인증 헤더와 요청 본문을 구성하는 데 어려움이 있어, 필요한 구문과 HTTP 상태 및 JSON 응답을 출력하는 방법을 설명해 달라고 요청하였습니다.

AI 가 제안한 내용: fetch()에 요청 메서드와 URL 을 지정하고, Authorization 헤더에 Bearer 토큰을 첨부하는 방법을 안내하였습니다. 또한 수정할 내용을 JSON 으로 전송하고, 응답의 HTTP 상태와 JSON code, message 를 콘솔에 출력하여 결과를 확인하는 예시를 제안하였습니다.

실제 반영 위치: 브라우저 개발자 도구 콘솔에서 실행한 수정·삭제 테스트용 요청 코드와 보고서의 '04 테스트 결과'에 반영하였습니다.

내가 확인하고 수정한 내용: 예시 코드의 요청 메서드, URL, 인증 헤더와 본문이 각각 어떤 역할을 하는지 확인하였습니다. 이후 사용 계정과 대상 게시글 ID를 실제 테스트 조건에 맞추고, 수정· 삭제 요청에 필요한 값으로 변경하여 실행하였습니다. 응답이 거부되었다는 사실만 확인하는 데서 끝내지 않고, 같은 게시글을 다시 조회하여 내용과 작성자 정보가 유지되는지 비교하였습니다.

검증 방법과 결과: T07 과 T08 에서 Bob 의 정상 토큰으로 Alice 의 글에 수정·삭제 요청을 보낸 결과, 각각 HTTP 403 과 JSON FORBIDDEN 이 반환되는 것을 확인하였습니다. 이후 공개 상세 조회로 같은 게시글을 다시 확인하였으며, 수정 요청 후에는 원문이 유지되고 삭제 요청 후에도 게시글이 남아 있는 것을 확인하였습니다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
6 주차_JWT_로그인과 인증.html	auth.js(backend-express/src/auth.js) - issueToken(), currentUserId() api.js(frontend-vue/src/api.js) - login(), authorizationHeaders()	JWT 발급과 검증, 브라우저 메모리 저장 및 Bearer 헤더 전송 방식을 참고하였습니다. 수업에서 설명한 요청 흐름을 바탕으로 서버 인증 처리와 프론트의 로그인 연결에 반영하였습니다.
7 주차 _JWT_접근제어와_Swagger.html	auth.js(backend-express/src/auth.js) - assertOwner()	인증된 사용자와 DB 작성자를 비교하는 권한 검사 방식과 401 과 403 의 차이를 참고하였습니다. 이를 작성자 권한 검사에 적용하고, 타인의 변경 요청이 거부된 뒤 원문이 유지되는지 확인하였습니다.

웹서비스설계 06

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorId 를 그대로 사용하면 사용자가 다른 사람의 ID 를 넣어 작성자를 위조할 수 있습니다. 이를 방지하기 위해 currentUserId()로 검증한 사용자 ID 를 서버에서 작성자로 지정하였습니다. 또한 fields()에서 제목과 본문만 허용하여, authorId 나 userId 같은 추가 필드는‘ 400 BAD_INPUT’으로 거부하도록 하였습니다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

수정 버튼을 숨겨도 Console 이나 API 클라이언트에서 수정, 삭제 요청을 직접 보낼 수 있으므로 서버의 권한 검사가 필요합니다. 제 코드에서는 assertOwner()가 인증된 사용자 ID 와 DB 에서 조회한 게시글의 작성자 ID 를 비교합니다. 두 값이 다르면 DB 를 변경하기 전에 ‘403 FORBIDDEN’을 반환하도록 하였습니다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰의 내용을 읽는 기능이며, 서명이 올바른지 검증하지 않으므로 그 결과만으로 사용자를 신뢰할 수 없습니다. 제 코드는 jwt.verify()로 HS256 알고리즘, 서명, 만료 여부, 발급자와 사용 대상을 검증하도록 하였습니다. 이후 exp 와 sub 의 형식을 확인하고, findPublic()으로 실제 계정이 존재하는지도 검사하였습니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 메모리의 토큰과 사용자 정보, 화면의 로그인 상태와 작성 중인 입력을 초기화합니다. 새로고침도 메모리의 로그인 정보를 초기화하지만, DB 에 저장한 게시글은 유지됩니다. 토큰이 만료되면 이후 보호 요청을 서버가 ‘401 TOKEN_EXPIRED’로 거부하며, 프론트는 이 응답을 받아 로그인 상태를 해제합니다. 이미 복사한 정상 토큰은 로그아웃이나 새로고침으로 서버에서 무효화되지 않으므로, 만료 전까지 사용할 수 있습니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

가장 어려웠던 부분은 토큰이 만료되었을 때 서버의 인증 결과와 화면의 로그인 상태를 맞추는 과정이었습니다. 요청이 실패하였을 때 오류만 전달하면 메모리에 저장된 토큰과 사용자 정보가 남을 수 있어, API 요청 처리와 화면 갱신 흐름을 함께 확인하였습니다. 이를 해결하기 위해 request() 함수에서 HTTP 401 을 받으면 clearSession()을 호출하도록 수정하고 run() 함수에서 사용자 정보를 다시 읽는 기존 구조와 연결하여 화면에도 로그아웃 상태가 반영되도록 하였습니다. 이후 토큰의 TTL 을 짧게 설정하고 만료될 때까지 기다린 뒤, 화면에서 내 정보 조회를 다시 요청하였습니다. T10 에서 ‘401/TOKEN_EXPIRED’가 반환되고, 로그인 시간 만료 안내와 함께 로그아웃 버튼이 사라지는 것을 확인하였습니다. 이를 통해 인증 오류를 처리할 때 서버의 응답뿐만 아니라 메모리의 사용자 정보와 화면 표시까지 함께 확인해야 한다는 점을 배웠습니다.

6. 현재 남은 문제와 다음에 개선할 점

현재는 제목과 내용을 중심으로 맛집 정보를 공유하도록 하였습니다. 이후에는 화면 디자인과 버튼 배치를 개선하여 게시글을 더 편리하게 읽고 작성할 수 있도록 하고자 합니다. 그리고 사진 첨부 기능을 추가하여 음식 사진을 함께 공유하고, 글만으로는 전달하기 어려운 특징을 보여줄 수 있도록 개선하고 싶습니다.

제출 확인

[V] 실제 코드와 설명 일치 [V] 테스트 직접 수행 [V] AI 활용 및 출처 기록 [V] 비밀 키와 토큰 제거

GitHub 확인

[V] 루트 README 작성 [V] 보고서 PDF 와 증거 업로드 [V] README 문서 링크 확인 [V] 교수자 접근 가능 [V] 저장소 URL 과 커밋 SHA 제출